UNIT 4 · PROJECT 03 · TECHNICAL REPORT

PatientWatch

Smart Healthcare Patient Deterioration Monitoring and Early Warning System

Research and education only. All records are synthetic. Not for diagnosis, triage, treatment or clinical monitoring.

1. Abstract and objective

Repeated vital signs can reveal evolving latent health conditions that are difficult to interpret from isolated readings. We implement an end-to-end prototype using a three-state Gaussian hidden Markov model (HMM), causal filtering, retrospective Forward–Backward smoothing, and Viterbi decoding. A read-only dashboard visualizes trajectories and research alert scores. This submission uses the synthetic-data option explicitly permitted by the assignment; no MIMIC-IV data or real patient records are used.

On 36 chronologically held-out synthetic patients, window-level AUROC is 0.792, sensitivity 72.2%, specificity 84.3%, and median lead time 5.5 hours among detected events. These numbers do not demonstrate clinical usefulness. All 12 test events were detected within the chosen horizon, while 14/24 non-event patients received an alert.

2. Dataset and sequence construction

The seed-42 simulator creates 180 patient admissions with 48 hourly measurements each (8,640 rows). Features are heart rate (bpm), oxygen saturation (%), respiratory rate (/min), systolic blood pressure (mmHg), and temperature (C). A three-state stationary Markov process generates stable, watch and critical regimes. Gaussian observation noise and patient-specific offsets vary measurements. Cell missingness is 7.5%; occasional 999 values emulate sensor corruption. Each patient has a new ID and admission date, spaced three days apart. The generator and raw CSV are included.

Split	Admissions	Purpose
Training	108 (60%)	Preprocessing, HMM, baseline
Validation	36 (20%)	Freeze alert thresholds
Test	36 (20%)	Final metrics and interpretation

3. Cleaning and leakage prevention

Range checks invalidate HR outside 30–220, SpO2 outside 65–100, respiratory rate outside 4–65, SBP outside 50–240, and temperature outside 33–43. Missing or invalid cells are forward-filled within patient only. Leading gaps use training medians. Training-fitted standardization is applied unchanged to validation and test. There is no backward fill or future interpolation. Lengths passed to the HMM enforce independent sequences; messages never cross patient boundaries. Admissions and their observations are chronologically ordered with no split overlap.

4. Model formulation and estimation

For latent regime z_t and standardized vital vector x_t, the joint density is pi(z_0) times the product of transition factors A(z_(t-1),z_t) and emission factors N(x_t; mu_(z_t), diag(var_(z_t))). There are three states with diagonal Gaussian emissions. Parameters are learned by Baum–Welch expectation-maximization (maximum likelihood), using hmmlearn with 150 maximum iterations, tolerance 0.001, and three deterministic restarts. The restart with greatest training likelihood is retained. State count and severity interpretation are fixed before test evaluation.

To address label permutation, states are ranked by learned standardized emission means: HR - SpO2 + respiratory rate - SBP + temperature. The resulting stable/watch/critical names are interpretive, not clinical diagnoses. Simulator labels are used solely to define evaluation outcomes, never to train the HMM or map state names.

5. Core algorithm: scaled Forward–Backward

Implemented from scratch in patientwatch/inference.py. Let log_b(t,k) be the log Gaussian density and s_t its row maximum. Set b_t = exp(log_b_t - s_t). Initialize alpha_0 = pi * b_0 and normalize by c_0. Recur alpha_t = (alpha_(t-1) A) * b_t / c_t, where c_t is the unnormalized row sum. This filter uses only past and present observations. Initialize beta_(T-1) = 1 and recurse beta_t = A[b_(t+1) * beta_(t+1)] / c_(t+1). Normalize gamma_t = alpha_t * beta_t to obtain a retrospective posterior. Restore log likelihood as sum_t(log c_t + s_t). Complexity is O(T K squared), with O(T K) storage. This is chain-factor-graph sum-product.

Viterbi is also implemented from scratch: maximize log transition plus previous best score for each state, add log emissions, save backpointers, then backtrack. It uses future sequence evidence in the final decoded path and is strictly retrospective. Neither smoothed gamma nor Viterbi paths enter early-warning scores.

Reference checks over all 180 sequences: max posterior error 5.551e-16; max log-likelihood error 1.705e-13, relative to hmmlearn. Unit tests also compare against exhaustive enumeration on a tiny chain, check a 10,000-step sequence for numerical stability, and perturb future observations to verify causal prefix invariance.

6. Alert protocol and outcome definition

A synthetic deterioration event is the first pair of consecutive simulator-critical hours, with onset at the first hour. Only windows strictly before the first event are scored. A window is positive if onset occurs within its next six hours. Negative windows without six hours of follow-up are censored; all post-event windows are excluded. This is simulator-ground-truth regime detection, not a validated clinical endpoint.

The research score is the probability of currently occupying or reaching the learned critical regime within six transitions. Make the critical row absorbing in transition matrix B and compute alpha_t B^6 e_critical. This is a surrogate reachability score; it is not the exact probability of the two-hour event, and it is not a calibrated clinical risk. A threshold is selected by maximum Youden J on validation, then frozen before evaluating test. Threshold comparisons are inclusive.

Hourly sensitivity = TP/(TP+FN); specificity = TN/(TN+FP). AUROC assesses score ranking across eligible windows. Event detection requires an above-threshold alert in the six hours strictly before onset. Lead time is onset minus earliest alert in that horizon; the median is over detected events only. Event recall and alert burden among non-event patients are reported separately. Whole-patient bootstrap resampling (400 replicates) accounts for within-patient dependence in the AUROC interval.

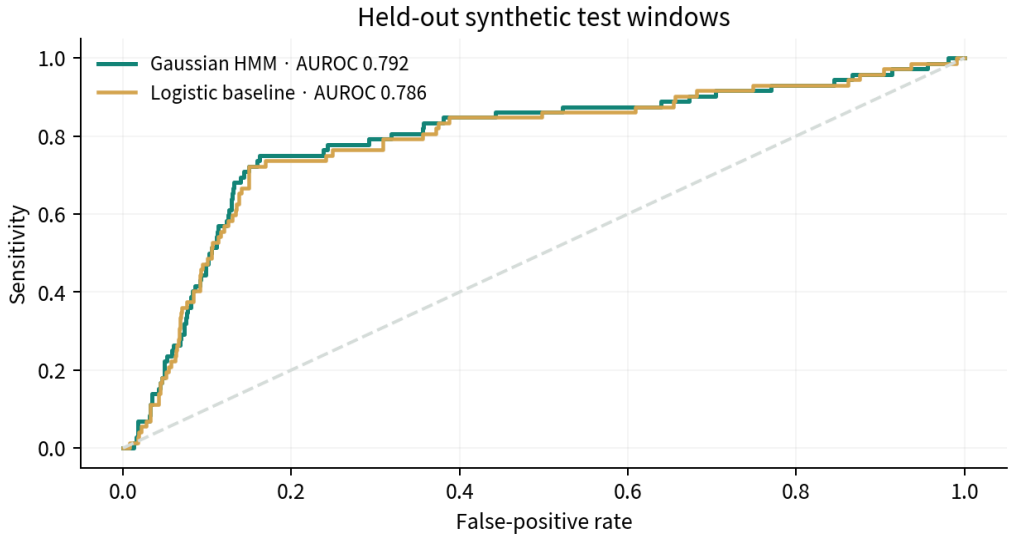
7. Results and baseline comparison

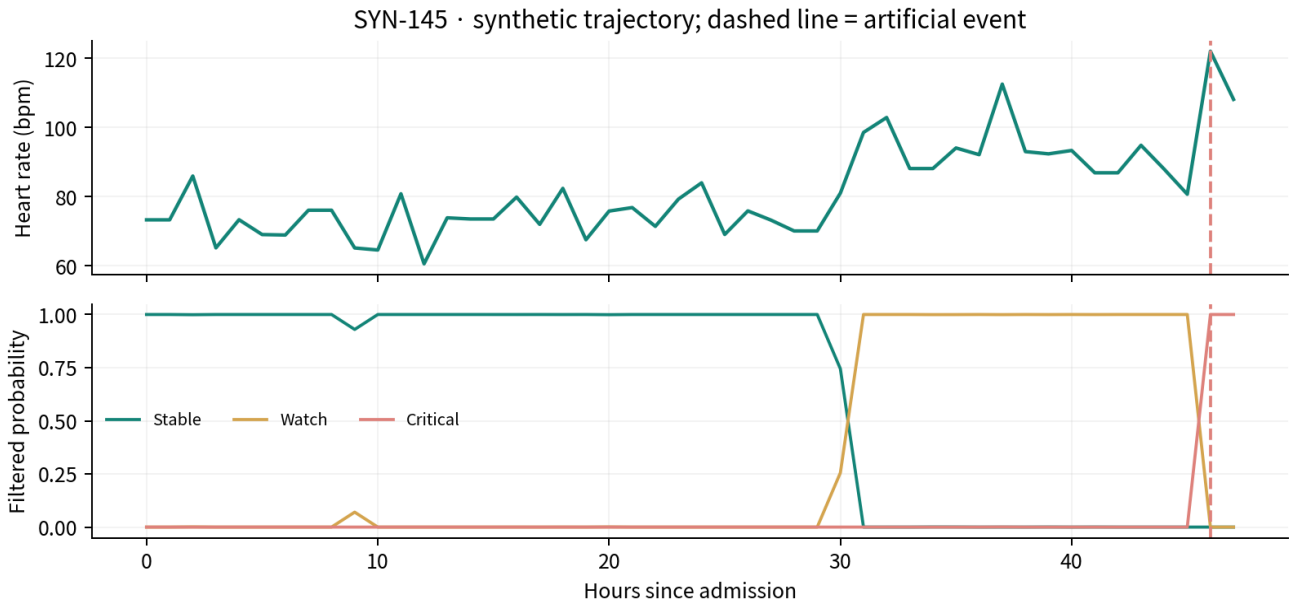
Metric	Gaussian HMM	Logistic baseline
AUROC	0.7925	0.7858
Sensitivity	72.22%	72.22%
Specificity	84.28%	83.37%
Brier score	0.04789	0.04882
Validation threshold	0.05945	0.06646

Test evaluation includes 1293 eligible windows, 72 positive. Confusion counts: TP=52, FP=192, TN=1029, FN=20. The 95% patient-bootstrap AUROC interval is [0.708, 0.875]. Event recall is 12/12, median detected-event lead time 5.5 hours. Alerted non-event patients: 14/24.

The supervised logistic baseline uses current causally cleaned and scaled vitals with training labels only, with a separate validation-selected threshold. The small AUROC difference is not a demonstrated significant advantage. This experiment shows implementation correctness and the trade-off between recall and false alerts, not clinical effectiveness.

8. Visualization and working demonstration





The Flask + vanilla JavaScript/SVG dashboard provides cohort state counts, patient search, all five vital charts, raw/cleaned comparison, hour selection, sequence replay, filtered probabilities, optional retrospective smoothing, Viterbi paths, synthetic event markers, a performance view, model transition matrix and submission downloads. Replay is a retrospective animation, not a live patient feed. State names and scores are labeled as research outputs. The application deliberately accepts no patient uploads.

9. Limitations and future work

The synthetic generator closely resembles HMM assumptions, potentially favoring the model. The cohort is small and stationary, with mostly random missingness. It omits treatments, demographic variation, confounding, irregular sampling, clinical adjudication and dataset shift. Artificial event labels depend on simulator regimes, not patient outcomes. Small baseline differences do not establish superiority. Scores are not calibrated to real-world prevalence. High event recall coexists with frequent alerts in non-event patients. No clinical or fairness claims are warranted.

A real-data extension would require independent credentialed MIMIC-IV access and approved handling, a clinically justified endpoint, outcome-blinded preprocessing, temporal and external validation, calibration, subgroup audits, alarm fatigue analysis and clinician review. This prototype is not intended for patient care. The project-specific objectives do not require a linear dynamical system, so no unnecessary Kalman component is added.

10. Reproducibility and submission map

Python 3.13. Install requirements.txt in a virtual environment, run python -m patientwatch.pipeline, then python -m pytest -q and python app.py. Open http://localhost:3000. Run python scripts/build_submission.py to execute the notebook and regenerate this report and the presentation from actual results. The README includes full commands, data schema, safety notes, API routes and a demonstration script. Seed 42 is fixed. Exact numerical last digits can vary with BLAS/platform; artifacts preserve the submitted run.

References

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. Proceedings of the IEEE, 77(2), 257–286. doi:10.1109/5.18626.
hmmlearn documentation: https://hmmlearn.readthedocs.io/
scikit-learn model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html
MIMIC-IV access information (not used): https://physionet.org/content/mimiciv/